



Apertura: viernes, 24 de julio de 2026, 00:00

Cierre: lunes, 27 de julio de 2026, 15:00

PROYECTO DE INTEGRACIÓN

SISTEMA DE GESTIÓN DE PARQUEADEROS SAAS MULTITENANT

1. Generalidades

Se solicita el diseño, desarrollo e implementación de un **Sistema de Gestión de Parqueaderos** bajo arquitectura de microservicios, modelo **SaaS (Software as a Service) multitenant**, donde cada tenant representa una empresa independiente con datos y configuraciones aislados. El sistema permitirá la administración integral de parqueaderos, incluyendo control de acceso, gestión de espacios, vehículos, usuarios y auditoría de eventos.

2. Alcance y Objetivo

Objetivo General:

Desarrollar una plataforma robusta, escalable y segura para la gestión de parqueaderos que permita a múltiples empresas (tenants) operar de forma independiente bajo una única infraestructura compartida.

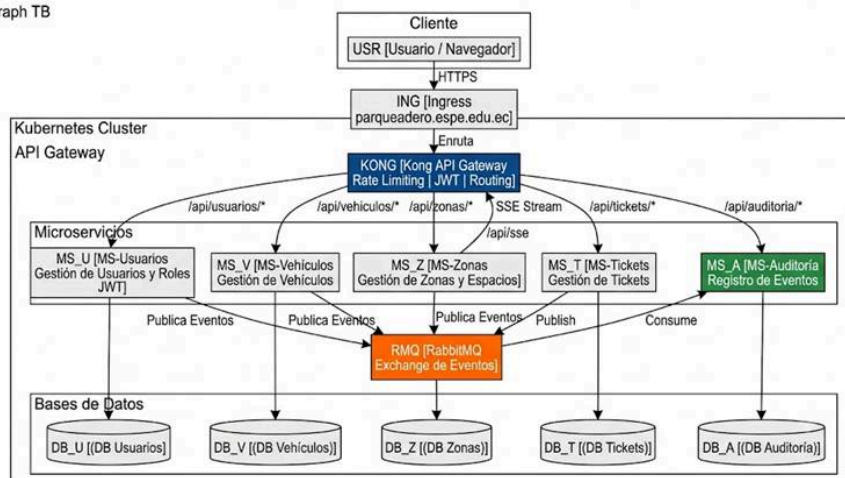
Objetivos Específicos:

- Implementar autenticación y autorización centralizada mediante JWT.
- Gestionar el ciclo de vida de vehículos, espacios y tickets de parqueo.
- Proveer comunicación asíncrona entre microservicios mediante RabbitMQ.
- Registrar y auditar todas las acciones realizadas en el sistema.
- Exponer los servicios a través de un API Gateway (Kong).
- Proveer una interfaz de usuario moderna con notificaciones en tiempo real (SSE).
- Automatizar el despliegue mediante contenedores Docker y orquestación con Kubernetes.
- Garantizar calidad de código mediante integración continua, análisis estático y notificaciones automatizadas.

3. Arquitectura de Microservicios

El sistema estará compuesto por los siguientes microservicios independientes, cada uno con su propia base de datos y Dockerfile:

graph TB



3.1. Microservicio de Gestión de Usuarios y Roles (MS-Usuarios)

- **Responsabilidad:** Administración de usuarios, perfiles, roles y permisos.
- **Autenticación:** Implementación de autenticación stateless mediante **JSON Web Tokens (JWT)**.
- **Funcionalidades:** Registro, login, asignación de roles, recuperación de contraseñas y control de acceso basado en roles (RBAC).
- **Consideración Multitenant:** Cada usuario debe pertenecer a un tenant específico, garantizando el aislamiento de datos entre empresas.

3.2. Microservicio de Vehículos (MS-Vehículos)

- **Responsabilidad:** Gestión del registro y catálogo de vehículos asociados a los usuarios de cada tenant.
- **Funcionalidades:** CRUD de vehículos, asociación con propietarios, tipos de vehículos (automóvil, motocicleta, camión, etc.) y placas.

3.3. Microservicio de Zonas y Espacios (MS-Zonas)

- **Responsabilidad:** Administración de la distribución física del parqueadero.
- **Funcionalidades:** Creación de zonas, definición de espacios de parqueo, estados de ocupación (disponible, ocupado, reservado, mantenimiento) y capacidad por zona.
- **Eventos en Tiempo Real:** Debe emitir eventos de cambio de estado para consumo vía SSE.

3.4. Microservicio de Gestión de Tickets (MS-Tickets)

- **Responsabilidad:** Control de entrada, salida y cobro de vehículos en el parqueadero.
- **Funcionalidades:** Generación de tickets, cálculo de tarifas, registro de tiempo de estacionamiento, estados del ticket (activo, pagado, vencido) e integración con el MS-Vehículos y MS-Zonas.

3.5. Microservicio de Auditoría (MS-Auditoría)

- **Responsabilidad:** Registro centralizado de todas las acciones y eventos generados por los demás microservicios.
- **Comunicación:** Debe consumir mensajes de una cola **RabbitMQ** a la cual publicarán eventos el resto de los microservicios.
- **Funcionalidades:** Almacenamiento de logs estructurados, trazabilidad de operaciones (quién, qué, cuándo, desde dónde), consulta de historial de eventos por tenant.

4. Infraestructura y Comunicación

4.1. API Gateway — Kong

- Toda la comunicación externa hacia los microservicios debe pasar por **Kong API Gateway**.
- Responsabilidades: Enrutamiento de peticiones, rate limiting, autenticación de tokens JWT, transformación de solicitudes y balanceo de carga.
- Kong debe estar desplegado como parte del clúster de Kubernetes.

4.2. Message Broker — RabbitMQ

- Debe existir una instancia de **RabbitMQ** como broker de mensajes.
- Cada microservicio (excepto el de Auditoría) debe publicar eventos de dominio relevantes (creación, actualización, eliminación) a una cola de exchange.
- El MS-Auditoría se suscribirá a estas colas para persistir los eventos.

4.3. Server-Sent Events (SSE)

- El frontend debe consumir un endpoint SSE expuesto (preferiblemente a través del API Gateway) que notifique en tiempo real la **disponibilidad de espacios de parqueo** (cambios de estado: libre/ocupado).
- Este canal debe estar vinculado al tenant autenticado, mostrando únicamente la información correspondiente a la empresa del usuario logueado.

5. Frontend

- Desarrollo de una aplicación web moderna (SPA/SSR) que sirva como interfaz única para los usuarios del sistema.
- **Funcionalidades mínimas:** Dashboard de ocupación en tiempo real, gestión de usuarios, vehículos, zonas, espacios y tickets; panel de auditoría (solo para roles administrativos).
- **SSE:** Integración de un componente de notificaciones en tiempo real que refleje la disponibilidad de espacios sin necesidad de recargar la página.
- **Multitenancy:** El frontend debe detectar el tenant (vía subdominio, header o path) y aislar visualmente y funcionalmente los datos de cada empresa.

6. Contenerización y Orquestación

6.1. Docker

- Cada microservicio, el frontend, Kong, RabbitMQ y cualquier componente adicional debe contar con su propio **Dockerfile**, garantizando la reproducibilidad del entorno.

6.2. Kubernetes (K8s)

- Dentro del repositorio monorepo debe existir una carpeta denominada **k8s/** que contenga todos los archivos de configuración en formato **YAML**, incluyendo pero no limitado a:
 - **Deployments** para cada microservicio y el frontend.
 - **Services** (ClusterIP, NodePort o LoadBalancer según corresponda).

- **ConfigMaps y Secrets** para variables de entorno y credenciales.
- **PersistentVolumeClaims** para bases de datos y logs si aplica.
- **Ingress** para la exposición del frontend.
- El despliegue debe permitir escalabilidad horizontal independiente de cada microservicio.

6.3. Ingress

- Se debe configurar un recurso **Ingress** que exponga el frontend bajo el dominio: **parqueadero.espe.edu.ec**
- El tráfico debe enrutarse a través del API Gateway o directamente al servicio del frontend según la estrategia de arquitectura definida, garantizando acceso seguro (TLS/HTTPS preferiblemente).

7. Repositorio y DevOps

7.1. Monorepo en GitHub

- Todo el código fuente (microservicios, frontend, configuraciones K8s, Dockerfiles, scripts) debe almacenarse en un **monorepo** en GitHub, organizado de manera modular por carpetas.

7.2. GitHub Actions

- Debe configurarse un pipeline de **CI/CD** mediante **GitHub Actions** que incluya:
 - Compilación y ejecución de pruebas unitarias e integración de cada microservicio.
 - Construcción y publicación de imágenes Docker a un registro de contenedores.
 - Análisis de calidad de código mediante **SonarCloud**.
 - Despliegue automatizado (o semiautomatizado) a Kubernetes.

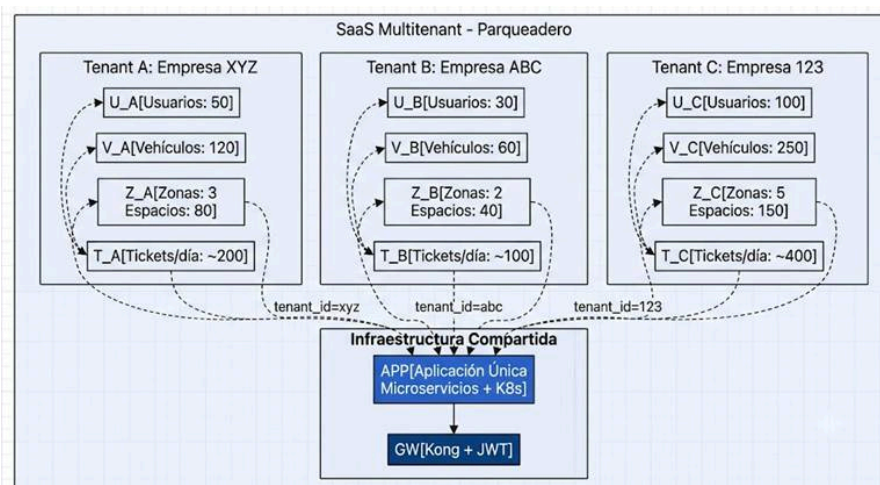
7.3. SonarCloud

- Integración obligatoria con **SonarCloud** para análisis estático de código, reporte de deuda técnica, vulnerabilidades de seguridad y cobertura de pruebas.
- Cada pull request debe ser validada por SonarCloud antes de su fusión.

7.4. Notificaciones de Telegram

- El pipeline de CI/CD debe incluir un paso de notificación a un bot de **Telegram** que informe:
 - Estado de los builds (éxito o fallo).
 - Resultados del análisis de SonarCloud.
 - Eventos de despliegue.

8. Modelo SaaS Multitenant



- La aplicación debe operar bajo un modelo **multitenant** donde cada empresa (tenant) es una entidad completamente independiente.
- **Aislamiento de datos:** Cada petición debe identificar el tenant (mediante JWT claims, headers o subdominio) y filtrar todas las consultas a base de datos para garantizar que un tenant no acceda a datos de otro.
- **Configuración por tenant:** Tarifas, horarios, zonas y capacidades deben ser configurables de forma independiente por cada empresa.
- **Escalabilidad:** La arquitectura debe permitir que nuevos tenants sean incorporados sin redistribuir la aplicación.

9. Entregables Esperados

1. Código fuente completo en monorepo de GitHub.
2. Dockerfiles funcionales para cada componente.
3. Carpeta k8s/ con manifiestos YAML para despliegue en Kubernetes.
4. Documentación de arquitectura y API (OpenAPI/Swagger).
5. Pipeline de CI/CD operativo en GitHub Actions.
6. Dashboard de SonarCloud con métricas de calidad.
7. Bot de Telegram integrado con notificaciones.
8. Frontend desplegado y accesible bajo parqueadero.espe.edu.ec.
9. Manual de despliegue y operación.

10. Consideraciones Finales

- Se prioriza la **seguridad** (JWT, secrets en K8s, comunicación segura).
- Se exige **trazabilidad completa** mediante el sistema de auditoría.
- El diseño debe ser **cloud-native**, aprovechando las capacidades de orquestación de Kubernetes.
- La solución debe ser **escalable** para soportar el crecimiento de tenants y volumen de transacciones.

Agregar entrega

Estado de la entrega

Estado de la entrega	Todavía no se han realizado envíos
Estado de la calificación	Sin calificar
Tiempo restante	1 día 2 horas restante

Regresar a Curso

Contactos - Aulas virtuales



(593) 23989400 extensión: 1571



[Mesa de ayuda](#)

[Resumen de retención de datos](#)

[Descargar la app para dispositivos móviles](#)